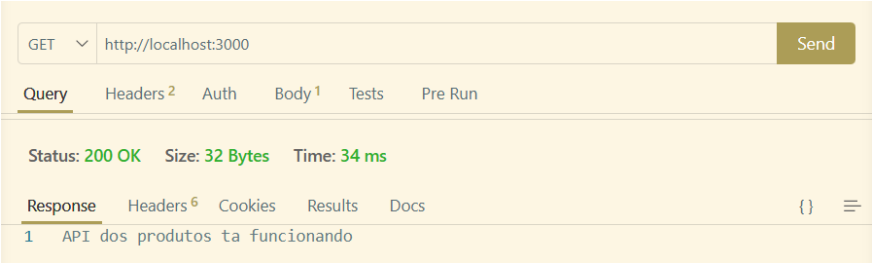
MÔNICA COTRIM MANFRINATO N°21 – FORMATIVA BACK END

Porque o Node.js com Express é adequado para esta API REST?

O Node.js é utilizado pois permite executar código Java Script fora do navegador (utilizando-o tanto no front quanto no back end), possuí alto desempenho e alta escalabilidade, além de ser assíncrono. Dentro do Node, O framework Express é adequado para API REST (Comunicação entre sistemas por meio do protocolo HTTP, com métodos padronizados) pois é mais simples na criação de comandos e rotas que realizam o CRUD (Criação, exibição, atualização e exclusão), sendo respectivamente o get, post, put e delete, fatores principais dentro de uma API REST. O Express recebe o pedido do front e chama o banco de dados, não exige estrutura de pastas específica – o que ajuda muito no desenvolvimento de APIs simples, como é a REST – e possuí uma curva de aprendizado menor quando comparado a outros frameworks que podem ser utilizados, como o NestJS e AdonisJS, que são mais complexos e robustos.

1. **GET** /: Rota inicial de teste.



2. **GET /produtos**: Listar todos os produtos (ordenados por ID decrescente).

GET http://localhost:3000/produtos Send

Query Headers 2 Auth Body 1 Tests Pre Run

Status: 200 OK Size: 227 Bytes Time: 17 ms

Response Headers 6 Cookies Results Docs


```
1  {
2    "sucesso": true,
3    "dados": [
4      {
5        "id": 2,
6        "nome": "Bala de chocolate",
7        "descricao": "Um pacote de 50g",
8        "preco": "10.00",
9        "disponivel": 0
10     },
11     {
12       "id": 1,
13       "nome": "Bala de coco",
14       "descricao": "Um pacote de 50g",
15       "preco": "10.00",
16       "disponivel": 1
17     }
18   ],
19   "total": 2
20 }
```


3. **GET /produtos/:id**: Buscar um produto específico (validar se o ID é numérico).

GET http://localhost:3000/produtos/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

Status: 200 OK Size: 128 Bytes Time: 15 ms

Response Headers 6 Cookies Results Docs


```
1  {
2    "sucesso": true,
3    "id": "1",
4    "dados": [
5      {
6        "id": 1,
7        "nome": "Bala de coco",
8        "descricao": "Um pacote de 50g",
9        "preco": "10.00",
10       "disponivel": 1
11     }
12   ]
13 }
```


4. **POST /produtos**: Cadastrar novo produto (validar campos obrigatórios e preço).

POST http://localhost:3000/produtos Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "nome": "Bala de chocolate",
3    "descricao": "Um pacote de 50g",
4    "preco": 10,
5    "disponibilidade": false
6  }
```

Status: 201 Created Size: 68 Bytes Time: 42 ms

Response Headers 6 Cookies Results Docs { }

```
1  {
2    "sucesso": true,
3    "mensagem": "Produto cadastrado com sucesso!",
4    "id": 2
5  }
```

5. **PUT /produtos/:id**: Atualizar produto (permitir atualização parcial e validar existência).

PUT http://localhost:3000/produtos/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
2      {
3        "nome": "Bala de coco fresco"
4      }
5
```

Status: 200 OK Size: 62 Bytes Time: 67 ms

Response Headers 6 Cookies Results Docs { }

```
1  {
2    "sucesso": true,
3    "mensagem": "Atualização feita com sucesso!"
4  }
```

GET http://localhost:3000/produtos/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
2       {
3         "nome": "Bala de coco fresco"
4       }
5
```

Status: 200 OK Size: 135 Bytes Time: 18 ms

Response Headers 6 Cookies Results Docs { }

```
1  {
2    "sucesso": true,
3    "id": "1",
4    "dados": [
5      {
6        "id": 1,
7        "nome": "Bala de coco fresco",
8        "descricao": "Um pacote de 50g",
9        "preco": "10.00",
10       "disponivel": 1
11     }
12   ]
13 }
```

6. **DELETE /produtos/:id**: Remover produto (validar existência antes de excluir).

DELETE http://localhost:3000/produtos/2 Send

Query Headers 2 Auth Body Tests Pre Run

Status: 200 OK Size: 45 Bytes Time: 36 ms

Response Headers 6 Cookies Results Docs { }

Copy

```
1 {
2   "sucesso": true,
3   "mensagem": "Produto apagado"
4 }
```

GET http://localhost:3000/produtos/2 Send

Query Headers 2 Auth Body Tests Pre Run

Status: 404 Not Found Size: 54 Bytes Time: 15 ms

Response Headers 6 Cookies Results Docs { }

```
1 {
2   "sucesso": false,
3   "mensagem": "Produto não encontrado"
4 }
```